

Tower Of Hanoi – Time Complexity through Substitution method.

From the given program we can say that :

```
void toh(int n, char src, char dest, char aux)
{
    if (n == 0)
    {
        return ;
    }

    toh(n - 1, src, aux, dest);

    cout << "Move " << n << " from " << src << " to " <<
dest << endl;

    toh(n - 1, aux, dest, src);
}
```

Diagram illustrating the time complexity analysis of the Tower of Hanoi program using the substitution method. The program is shown with annotations indicating the time taken by each recursive call and the base case.

- The base case `if (n == 0) { return ; }` is annotated with a box stating: *Takes 1 unit of time to run.*
- The recursive call `toh(n - 1, src, aux, dest);` is annotated with a box stating: *Takes $T(n - 1)$ time to run.*
- The output statement `cout << "Move " << n << " from " << src << " to " << dest << endl;` is annotated with a box stating: *Takes 1 unit of time, when $n > 0$ during recursion.*
- The recursive call `toh(n - 1, aux, dest, src);` is annotated with a box stating: *Takes $T(n - 1)$ time to run.*

***To, start with , $n > 0$, $TOH(n - 1, src, aux, dest)$
runs : $T(n - 1)$ times and $TOH(n - 1, aux, dest, src)$
runs $T(n - 1)$ times. With Base case running at least
once as recursion ends .***

When $n = 0$, it runs constant time i. e $T(0) = 1$.

Hence, Base Case:

$$T(0) = 1$$

And when $n > 0$,

$$T(n) = T(n-1) + T(n-1) + 1$$

$$\text{i. e. } T(n) = 2T(n-1) + 1$$

Here + 1 for printing the movement of the disk.

```
cout << "Move " << n << " from " << src << " to " <<
dest << endl;
```

Hence we get the Recursive relation as:

$$T(n) = \begin{cases} 1 & , \text{if } n = 0 \\ 2T(n-1) + 1 & , \text{if } n > 0 \end{cases}$$

Hence solving this recursive relation by substitution method we get:

$$T(n) = 2T(n - 1) + 1$$

$$\text{Now, } T(n - 1) = 2T(n - 2) + 1$$

Substituting $T(n - 1) = 2T(n - 2) + 1$ in $T(n)$ we get:

$$T(n) = 2[2T(n - 2) + 1] + 1$$

$$\Rightarrow 2^2T(n - 2) + 2 + 1$$

$$\text{Again, } T(n - 2) = 2T(n - 3) + 1$$

Substituting $T(n - 2) = 2T(n - 3) + 1$ in $T(n)$ we get:

$$T(n) = 2^2T(n - 2) + 2 + 1$$

$$\Rightarrow 2^2[2T(n - 3) + 1] + 2 + 1$$

$$\Rightarrow 2^3T(n - 3) + 2^2 + 2 + 1$$

Now, if it runs upto `k` times we get:

$$T(n) = 2^kT(n - k) + 2^{k-1} + 2^{k-2} \dots + 2^2 + 2 + 1$$

As, our base case is $T(0)$ not $T(1)$, hence:

$n - k = 0$, therefore, $k = n$, we get:

$$T(n) = 2^nT(n - n) + 2^{n-1} + 2^{n-2} + \dots + 2^2 + 2 + 1$$

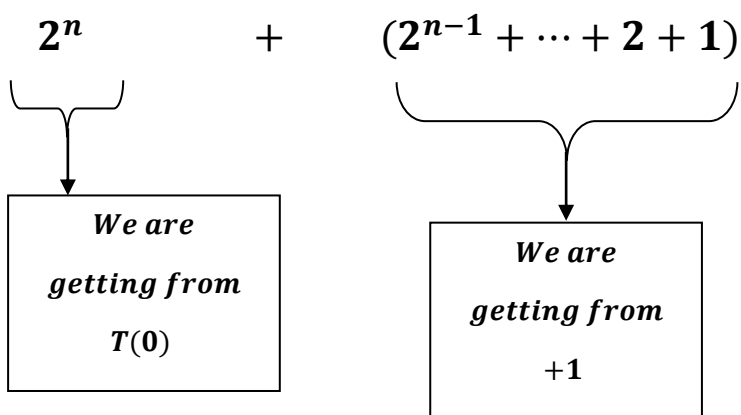
$$T(n) = 2^n T(0) + (2^{n-1} + 2^{n-2} + \dots + 2^2 + 2 + 1)$$

Substituting $T(0) = 1$, we get:

$$\Rightarrow 2^n \times 1 + 2^{n-1} + \dots + 2 + 1$$

$$\Rightarrow 2^n + (2^{n-1} + \dots + 2 + 1)$$

Now, conceptual scenario:



Hence cannot combine both conceptually:

Geometric progression(series) upto 'n' $\sum_{i=0}^n ar^i$:

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^n = a \left(\frac{r^{n+1} - 1}{r - 1} \right), \text{ where}$$

S(n) is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

Similarly,

Geometric progression(series) upto n - 1 $\sum_{i=0}^{n-1} ar^i$:

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^{n-1} = a \left(\frac{r^n - 1}{r - 1} \right), \text{ where}$$

S(n) is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

We get:

$$\Rightarrow 2^n + \left\{ 1 \left(\frac{2^n - 1}{2 - 1} \right) \right\}$$

$$\Rightarrow 2^n + (2^n - 1)$$

$$\Rightarrow 2^{n+1} - 1$$

putting Big O we get:

$$\Rightarrow O(2^{n+1} - 1)$$

$$\Rightarrow O(2^{n+1}) - O(1)$$

$$\Rightarrow O(2^{n+1})$$

$$\Rightarrow O(2^n \times 2)$$

$$\Rightarrow O(2^n) \times O(2)$$

$$\Rightarrow O(2^n \times 2)$$

$$\Rightarrow O(2^n).$$
